gemini-conversation

Exported on: 6/23/2026, 6:27:27 PM

User

what is the c++ fsm library that defines state<T> and event<E> in a state transition table

Gemini

The C++ finite state machine library you are looking for is `[Boost::ext].SML` (State Machine Language), developed by Kris Jusiak.

GitHub

It is a modern, single-header C++14/17/20 library featuring an embedded domain-specific language (eUML-like DSL) to build declarative state transition tables at compile-time without macros.

Atadiat

Key Syntax Elements

In `Boost.SML`, states and events are explicitly wrapped or represented as:

- `sml::state<T>` (or using the `_s` string literal suffix like `"my_state"_s`)
- `sml::event<E>` ### Example Transition Table A typical state transition table in `Boost.SML` binds these together using intuitive operators (`+`, `[]`, `/`, `=`) representing `src_state + event [ guard ] / action = dst_state`:

  boost-ext.github.io

C++

```cpp
#include <boost/sml.hpp>
#include <iostream>

namespace sml = boost::sml;

// Define custom events
struct connect {};
struct disconnect {};

// Guards and Actions (can be lambdas or callables)
auto is_valid = [] { return true; };
auto send_ack = [] { std::cout << "Sending ACK...\n"; };
```

```cpp
struct connection_fsm {
    auto operator()() const noexcept {
        using namespace sml;

        return make_transition_table(
            // * indicates the initial state
            *"Disconnected"_s + event<connect> [ is_valid ] / send_ack = "Connected"_s,
             "Connected"_s    + event<disconnect>                     = "Disconnected"_s
        );
    }
};

int main() {
    sml::sm<connection_fsm> sm;
    sm.process_event(connect{});
}
```

Highlights of `Boost.SML`

- **Zero Overhead:** It doesn't use dynamic memory allocation or virtual functions, meaning its runtime speed is as fast as a hand-coded `switch` statement.
- **Compile-time Checking:** Transitions, guards, and action dependencies are verified entirely at compile time.
- **No Dependencies:** Despite the "Boost" moniker, `[Boost::ext].SML` is completely standalone and does not require the rest of the Boost library collection.

User

explain the syntax of the transition table code

Gemini

The transition table syntax in `Boost.SML` is designed to look exactly like a formal finite state machine state-diagram. It uses C++ operator overloading (`+` , `[]` , `/` , `=`) to form a domain-specific language (DSL).

The fundamental rule for reading any line in the table is:

$$\text{Current State} + \text{Event} \ [\text{Guard}] \ / \ \text{Action} = \text{Next State}$$

Breaking Down the Operators

Here is exactly what each operator does in that line of code:

- `*` **(The Asterisk prefix):** Tells the library which state is the **initial starting state** when the machine boots up. You only use it once on your starting state (`*"Disconnected"_s`).
- `+` **(State + Event):** Binds an event to a specific state. It reads as: "If the machine is in this state **AND** this specific event occurs..."
- `[]` **(The Guard Condition):** A boolean check (like a lambda or function) that runs when the event fires. The transition only proceeds if this returns `true` . If it returns `false` , the event is ignored, and the machine stays put.
- `/` **(The Action Trigger):** The side effect or function that executes during the transition. This is where you put your logic (e.g., sending a network packet, logging data).
- `=` **(The Target State Assignment):** Defines the final state the machine shifts into if the guard passes and the action finishes successfully.

Step-by-Step Translation of the Example

Let's look at the two lines from the transition table side-by-side with plain English:

C++

```
*"Disconnected"_s + event<connect> [ is_valid ] / send_ack = "Connected"_s
```

English: Start the machine in the **Disconnected** state. If a **connect** event happens, check if `is_valid` is true. If it is, execute the `send_ack` function and move the machine into the **Connected** state.

C++

```
"Connected"_s + event<disconnect> = "Disconnected"_s
```

> **English:** If the machine is in the **Connected** state and a **disconnect** event happens, immediately move back to the **Disconnected** state. (Notice that guards `[]` and actions `/` are entirely optional!)

A Quick Note on State Representation

You probably noticed two different ways states are written in SML:

1. `"StateName"_s` : This uses C++ user-defined literal strings to create states completely on the fly without defining classes for them.
2. `state<my_class>` : If your state needs to hold data or have a distinct type structure, you wrap that class name inside `state<T>`. Both methods work identically within the operator syntax.

User

how does use C++ user-defined literal strings to create states completely on the fly without defining classes for the

Gemini

To understand how `Boost.SML` creates states on the fly using strings like `"my_state"_s`, we have to look at a feature introduced in C++11 and expanded in later standards: **User-Defined Literals (UDL)** combined with **Compile-Time String Parsing**.

Instead of treating `"my_state"_s` as a runtime `const char*`, the library tricks the compiler into turning that string into a **unique compile-time type**.

Here is a breakdown of how this magic works behind the scenes.

1. The Custom Literal Operator (`operator""_s`)

In C++, when you append a suffix to a literal (like `1.0f` for float or `10s` for chrono seconds), the compiler looks for a matching literal operator.

`Boost.SML` defines an operator that looks roughly like this:

C++

```
template <class T, T... Chars>
constexpr auto operator""_s() {
    return state<string_literal<Chars...>>{};
}
```

2. Turning Characters into a Unique Type

Normally, a string literal is just an array of characters in memory. However, by using a template pack (`T... Chars`), the compiler unpacks the string at compile time into individual character constants.

For example, when you write `"Idle"_s`, the compiler expands it into a distinct template instantiation:

C++

```
// What the compiler sees under the hood:
state<string_literal<'I', 'd', 'l', 'e'>>
```

Because every unique string creates a unique combination of template arguments, `"Idle"_s` and `"Running"_s` become **two completely different types** at compile time.

Why do this instead of using regular classes?

This approach offers the best of both worlds: the convenience of strings and the performance of types.

- **Zero Runtime Overhead:** Because the parsing happens entirely during compilation, the string `"Idle"` doesn't actually exist as a runtime string variables array inside the state machine. There are no string comparisons (`strcmp`) happening when events fire.
- **Compile-time Safety:** If you typo a state name in your transition table, the compiler treats it as a brand-new unique type. While it won't catch typos instantly, it ensures that your transition table remains highly optimized and strictly typed without you needing to write boilerplate like:

C++

```
struct IdleState {};
struct RunningState {};
```

Summary of the Workflow

```
      Your Code:  "Idle"_s
                      |
                      ▼ (Compile-time)
Literal Operator:  operator""_s<'I', 'd', 'l', 'e'>()
                      |
```

```
                        ▼ (Type Generation)
  Resulting Type:  sml::state<sml::aux::string<'I', 'd', 'l', 'e'>>
```

This type is then used by the compiler to map out the finite state machine's state transition array, giving you clean code without a performance penalty.

User

what is the code that defines state, event, guard, and action. like a whole example in a single file

Gemini

Here is a complete, self-contained example in a single file. This example models a simplified **Smart Thermostat** that manages a heating element based on temperature readings.

It demonstrates how to define distinct classes/structs for **States**, **Events**, **Guards**, and **Actions**, and how to tie them all together using `Boost.SML`.

C++

```cpp
#include <boost/sml.hpp>
#include <iostream>

namespace sml = boost::sml;

// ==========================================
// 1. STATES
// ==========================================
// If a state doesn't need data, we can use empty structs.
struct Off {};
struct Idling {};
struct Heating {};

// ==========================================
// 2. EVENTS
// ==========================================
// Events can be empty triggers, or they can carry data payload.
struct TurnOn {};
struct TurnOff {};

struct TemperatureUpdate {
    double current_temp;
```

```
    double target_temp;
};

// ============================================
// 3. GUARDS
// ============================================
// Guards evaluate a condition and return a boolean.
// They can inspect data passed via events.
struct IsTooCold {
    bool operator()(const TemperatureUpdate& event) const noexcept {
        return event.current_temp < (event.target_temp - 1.0); // 1 degree buffer
    }
};

struct IsWarmEnough {
    bool operator()(const TemperatureUpdate& event) const noexcept {
        return event.current_temp >= event.target_temp;
    }
};

// ============================================
// 4. ACTIONS
// ============================================
// Actions are side-effects executed during a transition.
struct StartHeater {
    void operator()() const noexcept {
        std::cout << "[Action] -> Turning heating coils ON.\n";
    }
};

struct StopHeater {
    void operator()() const noexcept {
        std::cout << "[Action] -> Turning heating coils OFF.\n";
    }
};
```

```cpp
struct LogShutdown {
    void operator()() const noexcept {
        std::cout << "[Action] -> System safely powered down.\n";
    }
};

// ========================================
// 5. THE STATE MACHINE DEFINITION
// ========================================
struct ThermostatFSM {
    auto operator()() const noexcept {
        using namespace sml;

        // Instantiate our Guards and Actions
        auto is_too_cold    = IsTooCold{};
        auto is_warm_enough = IsWarmEnough{};
        auto start_heater   = StartHeater{};
        auto stop_heater    = StopHeater{};
        auto log_shutdown   = LogShutdown{};

        return make_transition_table(
            // * marks the initial state
            // Src State + Event                    [ Guard ]        / Action       = Dst State
            * state<Off> + event<TurnOn>                                             = state<Idling>,

              state<Idling> + event<TemperatureUpdate> [ is_too_cold ]  / start_heater = state<Heating>,
              state<Idling> + event<TurnOff>                            / log_shutdown = state<Off>,

              state<Heating> + event<TemperatureUpdate> [ is_warm_enough ] / stop_heater  = state<Idling>,
              state<Heating> + event<TurnOff>                            / (stop_heater, log_shutdown) = state
              // Note: You can sequence multiple actions using a comma inside parenthesis ^
        );
    }
};
```

```cpp
// ========================================
// MAIN EXECUTION
// ========================================
int main() {
    // Instantiate the State Machine
    sml::sm<ThermostatFSM> thermostat;

    std::cout << "--- Booting Thermostat ---\n";

    // 1. Turn the system on
    std::cout << "\nSending TurnOn event...\n";
    thermostat.process_event(TurnOn{}); // Moves Off -> Idling

    // 2. Simulate temperature drop (Target: 22C, Current: 19C)
    std::cout << "\nSending Temp Update (19°C, Target 22°C)...\n";
    thermostat.process_event(TemperatureUpdate{19.0, 22.0}); // Moves Idling -> Heating (Triggers start_heater

    // 3. Simulate temperature rise (Target: 22C, Current: 22.5C)
    std::cout << "\nSending Temp Update (22.5°C, Target 22°C)...\n";
    thermostat.process_event(TemperatureUpdate{22.5, 22.0}); // Moves Heating -> Idling (Triggers stop_heater)

    // 4. Shut the system down
    std::cout << "\nSending TurnOff event...\n";
    thermostat.process_event(TurnOff{}); // Moves Idling -> Off (Triggers log_shutdown)

    return 0;
}
```

How to Compile This

Because `Boost.SML` is a header-only library, you just need to download `sml.hpp` and point your compiler to it. It requires at least C++14.

Bash

```
g++ -std=c++14 main.cpp -I /path/to/sml/include -o thermostat
./thermostat
```